

■ CineVault: Guia Rápido & Prático

Do Java (POO / Camadas) ao TypeScript / Next.js: Async, API Externa e Banco Back4App

1. O Mapa Geral do Fluxo (Ciclo de Vida dos Dados)

Em um sistema de locadora moderno, o usuário não precisa digitar manualmente nome, diretor, gênero e ano de cada filme. O ciclo de dados divide-se em 3 etapas claras:

Etapa	O que acontece	Tecnologia Envolvida
1. Busca na API	Usuário digita 'Matrix'. O app consulta a base global de filmes (OMDb) e traz todos os metadados prontos em JSON.	HTTP GET / fetch assíncrono (OMDb API)
2. Cadastro no Banco	Com os dados preenchidos da API, o usuário confirma o cadastro. O filme é salvo no banco de dados da locadora com status 'DISPONÍVEL'.	SDK Back4App / Parse Server (POST/save)
3. Consulta / Listagem	A locadora recupera os filmes cadastrados para montar a vitrine e gerenciar locações / devoluções.	Parse Query / find() -> Tabela/Grid na tela

2. O que é 'Async'? O Choque Java vs JavaScript

Por que precisamos de assincronismo? Quando seu código pede algo pela rede (internet) ou para um banco de dados remoto, a resposta demora centenas de milissegundos. Se o código travasse esperando (síncrono bloqueante), o programa ou a aba do navegador congelaria por completo.

Analogia do Restaurante:

- **Síncrono (Ruim):** O garçom anota seu pedido e fica parado olhando a cozinha cozinhar até o prato sair. Ele não atende nenhuma outra mesa.
- **Assíncrono (Ideal):** O garçom passa o pedido à cozinha, te dá uma comanda (**Promise / CompletableFuture**) e continua atendendo outras mesas. Quando a cozinha avisa que o prato está pronto, ele te entrega.

A Comparação de Código: Como pedir dados sem travar

Java: Requisição Assíncrona (CompletableFuture)

```
// Em Java (HttpClient nativo assíncrono com CompletableFuture):
public CompletableFuture<String> buscarFilmeAsync(String titulo) {
    HttpClient client = HttpClient.newHttpClient();
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://www.omdbapi.com/?t=" + titulo + "&apikey=CHAVE"))
        .GET()
        .build();

    // sendAsync envia em thread secundária, NÃO trava a thread principal!
    return client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
        .thenApply(HttpResponse::body); // Extraí o corpo quando chegar
}

// O que cada função faz:
// 1. HttpClient.newHttpClient(): Instancia o cliente HTTP do Java.
// 2. client.sendAsync(...): Dispara a requisição em background e retorna uma promessa futura (CompletableFuture).
// 3. thenApply(...): Callback acionado assim que a resposta da rede chega.
```

JavaScript / TypeScript: async / await

```
// Em TypeScript / JavaScript (async / await nativo):
async function buscarFilmeAsync(titulo: string): Promise<any> {
    // 1. O 'await' pausa ESTA FUNÇÃO até a rede responder, mas NÃO trava o navegador
    const resposta = await fetch(`https://www.omdbapi.com/?t=${titulo}&apikey=CHAVE`);

    // 2. Extraí e converte o JSON que veio no corpo da resposta
    const dados = await resposta.json();

    return dados;
}

// O que cada função/palavra faz:
// 1. async: Declara que a função retorna uma Promise (equivalente ao CompletableFuture do Java).
// 2. fetch(): Faz o GET HTTP nativo do navegador/Node.
// 3. await: Suspende a execução da linha até a Promise resolver, entregando o resultado limpo.
```

3. Arquitetura em Camadas: Você já conhece isso do Java!

Se você está aprendendo Camadas em Java, observe como o projeto da locadora CineVault em Next.js usa **exatamente os mesmos conceitos** com nomes ligeiramente diferentes:

Camada	No Java	No CineVault (Next.js)	Responsabilidade
Modelo (Entidade)	Filme.java (Classe com atributos e getters/setters)	types/movie.ts (Interface TypeScript)	Define a estrutura de dados: título, diretor, ano, status.
Persistência (DAO/Repo)	FilmeDAO.java ou FilmeRepository.java	services/parseMovieService.ts	Quem fala com o banco: faz save(), find(), delete().
Integração Externa	OmdbClientService.java	services/omdbService.ts	Quem busca filmes na internet com fetch().
Apresentação (UI)	JSP / Swing / Thymeleaf	Componente React (page.tsx)	O formulário com botão e a tabela na tela.

4. Código Detalhado: As 3 Operações Principais

Abaixo estão os blocos essenciais de código que você precisa entender para a entrega do seu projeto.

Operação 1: Buscar Filme na API Externa (OMDb)

TypeScript: omdbService.ts

```
// src/services/omdbService.ts
export async function buscarFilmeNoOMDb(titulo: string) {
  const chave = process.env.NEXT_PUBLIC_OMDB_API_KEY;
  const url = `https://www.omdbapi.com/?apikey=${chave}&t=${encodeURIComponent(titulo)}`;

  const res = await fetch(url);
  const dados = await res.json();

  if (dados.Response === "False") {
    throw new Error(dados.Error || "Filme não encontrado no OMDb.");
  }

  // Retorna um objeto limpo e adaptado para a nossa locadora:
  return {
    title: dados.Title,
    director: dados.Director,
    year: parseInt(dados.Year),
    genre: dados.Genre,
    posterUrl: dados.Poster !== "N/A" ? dados.Poster : "",
    synopsis: dados.Plot
  };
}

// O QUE FAZ:
// - encodeURIComponent: Limpa espaços e acentos no nome para a URL não quebrar.
// - fetch(): Dispara a requisição GET assíncrona.
// - dados.Response === 'False': A OMDb responde 200 OK mesmo quando não acha o filme,
//   avisando com esse campo se encontrou ou não.
```

Operação 2: Cadastrar Filme no Banco de Dados (Back4App)

TypeScript: parseMovieService.ts - cadastrarFilme()

```
// src/services/parseMovieService.ts
import Parse from "@lib/parseClient";

export async function cadastrarFilme(dadosFilme: any) {
  // 1. Cria uma nova instância da classe 'Movie' no Back4App (como dar 'new Movie()' em Java)
  const Movie = Parse.Object.extend("Movie");
  const novoFilme = new Movie();

  // 2. Preenche os campos (equivalente aos setters: filme.setTitulo(...))
  novoFilme.set("title", dadosFilme.title);
  novoFilme.set("director", dadosFilme.director);
  novoFilme.set("year", dadosFilme.year);
  novoFilme.set("genre", dadosFilme.genre);
  novoFilme.set("posterUrl", dadosFilme.posterUrl);
  novoFilme.set("status", "DISPONIVEL"); // Regra de negócio: todo filme nasce disponível

  // 3. Salva no banco de dados remoto de forma assíncrona
  const resultado = await novoFilme.save();
  return resultado.id; // Retorna o ID único gerado pelo banco
}

// O QUE FAZ:
// - Parse.Object.extend('Movie'): Mapeia a tabela 'Movie' no banco NoSQL do Parse.
// - novoFilme.set(campo, valor): Atribui as propriedades ao objeto.
// - novoFilme.save(): Envia um POST para o Back4App persistir o registro.
```

Operação 3: Buscar / Listar Filmes do Banco (Back4App)

TypeScript: parseMovieService.ts - listarFilmes()

```
// src/services/parseMovieService.ts
export async function listarFilmes() {
  // 1. Cria uma consulta para a classe 'Movie' (equivalente ao SELECT * FROM Movie no SQL)
  const Movie = Parse.Object.extend("Movie");
  const query = new Parse.Query(Movie);

  // 2. Ordena os mais recentes primeiro
  query.descending("createdAt");

  // 3. Executa a busca assíncrona no banco de dados
  const resultados = await query.find();

  // 4. Converte os objetos Parse em um array JSON limpo para a tela
  return resultados.map(filme => ({
    id: filme.id,
    title: filme.get("title"),
    director: filme.get("director"),
    year: filme.get("year"),
    genre: filme.get("genre"),
    posterUrl: filme.get("posterUrl"),
    status: filme.get("status")
  }));
}

// O QUE FAZ:
// - new Parse.Query(Movie): Cria o construtor de consulta (Query Builder).
// - query.descending('createdAt'): Adiciona ordenação (ORDER BY criado_em DESC).
// - query.find(): Executa a query remota no Back4App.
// - filme.get('campo'): Equivalente ao getter do Java (filme.getTitulo()).
```

5. Como a Tela (React View) Conecta Tudo

Na interface do usuário, nós juntamos a busca da API com o cadastro no banco usando funções de clique (event handlers):

React / Next.js: Conexão dos Eventos na View

```
// Trecho dentro do componente React (src/app/page.tsx):
export default function CatalogoFilmes() {
  const [filmeBuscado, setFilmeBuscado] = useState(null);

  // PASSO A: Usuário clica em 'Buscar na API'
  async function handleBuscar(nomeDigitado: string) {
    const dados = await buscarFilmeNoOMDb(nomeDigitado);
    setFilmeBuscado(dados); // Mostra na tela para confirmação
  }

  // PASSO B: Usuário gostou e clica em 'Confirmar e Cadastrar na Locadora'
  async function handleCadastrar() {
    if (!filmeBuscado) return;
    await cadastrarFilme(filmeBuscado); // Salva no Back4App
    alert('Filme cadastrado com sucesso no banco!');
    carregarFilmesDoBanco(); // Atualiza a lista na tela
  }

  return (
    <div>
      {/* Inputs, botões e cards de filmes renderizados aqui */}
    </div>
  );
}
```

6. Resumo de Bolso para Defender seu Projeto

Se o professor ou colega perguntar:

- **Por que usou async/await?** Porque chamadas de rede para a OMDb API e para o Back4App levam tempo. Com async/await, o código aguarda a resposta sem congelar a interface do usuário.
- **Como organizou o código?** Em arquitetura de camadas: uma camada de Tipos (Model), uma camada de Serviços para o OMDb e Back4App (Repository/DAO) e a Camada de Interface em React (View).
- **O que é o Back4App?** É um Backend as a Service (BaaS) baseado em Parse Server que substitui a necessidade de criar controllers manuais e SQL para CRUD simples, persistindo objetos diretamente.